

DS	Session	Tree 2	Question Information	Level 1	Challenge 71
----	---------	--------	----------------------	---------	--------------

Question description

Given two rooted trees, your task is to find out if they are *isomorphic*, i.e., it is possible to draw them so that they look the same.

Constraints

- $1 \leq t \leq 1000$
- $2 \leq n \leq 10^5$
- the sum of all values of n is at most 10^5

Input

The first input line has an integer t : the number of tests. Then, there are t tests described as follows:
The first line has an integer n : the number of nodes in both trees. The nodes are numbered $1, 2, \dots, n$, and node 1 is the root.
Then, there are $n-1$ lines describing the edges of the first tree, and finally $n-1$ lines describing the edges of the second tree.

Output

For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
3
1 2
2 3
1 2
1 3
3
1 2
2 3
1 3
3 2
```

Test Case 2

INPUT (STDIN)

```
2
3
1 2
2 3
1 3
3 3
3
1 2
2 3
1 3
3 3
```

EXPECTED OUTPUT

NO
YES

EXPECTED OUTPUT

NO
NO

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

480

Test Case 3

NLOC

65

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <stdint.h>
5
6 typedef struct {
7     int *head;
8     int *to;
9     int *nxt;
10    int edge_cnt;
11} Graph;
12
13void init_graph(Graph *g, int n) {
14    g->head = (int*)malloc((n + 1) * sizeof(int));
15    g->to = (int*)malloc(2 * n * sizeof(int));
16    g->nxt = (int*)malloc(2 * n * sizeof(int));
17    for (int i = 0; i <= n; i++) g->head[i] = -1;
18    g->edge_cnt = 0;
19}
20
21void add_edge(Graph *g, int u, int v) {
22    g->to[g->edge_cnt] = v;
23    g->nxt[g->edge_cnt] = g->head[u];
24    g->head[u] = g->edge_cnt++;
25}
```

```

26
27void free_graph(Graph *g) {
28    free(g->head);
29    free(g->to);
30    free(g->nxt);
31}
32
33int cmp_u64(const void *a, const void *b) {
34    uint64_t x = *(const uint64_t*)a;
35    uint64_t y = *(const uint64_t*)b;
36    if (x < y) return -1;
37    if (x > y) return 1;
38    return 0;
39}
40
41static inline uint64_t splitmix64(uint64_t x) {
42    x += 0x9e3779b97f4a7c15ULL;
43    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
44    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;

```

```

45     return x ^ (x >> 31);
46}
47
48uint64_t dfs(int u, int p, Graph *g) {
49     uint64_t child_hashes[64];
50     uint64_t *arr = child_hashes;
51     int cap = 64;
52     int cnt = 0;
53
54     for (int e = g->head[u]; e != -1; e = g->nxt[e]) {
55         int v = g->to[e];
56         if (v == p) continue;
57         if (cnt >= cap) {
58             cap *= 2;
59
60             for (int i = 0; i < cnt; i++) new_arr[i] = arr[i];
61             if (arr != child_hashes) free(arr);
62             arr = new_arr;

```

```

63     }
64     arr[cnt++] = dfs(v, u, g);
65 }
66
67 qsort(arr, cnt, sizeof(uint64_t), cmp_u64);
68
69 uint64_t h = 14695981039346656037ULL;
70 for (int i = 0; i < cnt; i++) {
71     h ^= splitmix64(arr[i]);
72     h *= 1099511628211ULL;
73 }
74 h ^= (uint64_t)cnt;
75
76 if (arr != child_hashes) free(arr);
77 return h;
78}
79
80int main() {

```

```

81  int t;
82  if (scanf("%d", &t) != 1) return 0;
83  while (t--) {
84      int n;
85      if (scanf("%d", &n) != 1) break;
86      Graph g1, g2;
87      init_graph(&g1, n);
88      init_graph(&g2, n);
89
90      for (int i = 0; i < n - 1; i++) {
91          int u, v;
92          if (scanf("%d %d", &u, &v) == 2) {
93              add_edge(&g1, u, v);
94              add_edge(&g1, v, u);
95          }
96      }
97      for (int i = 0; i < n - 1; i++) {
98          int u, v;

```

```
99         if (scanf("%d %d", &u, &v) == 2) {
100             add_edge(&g2, u, v);
101             add_edge(&g2, v, u);
102         }
103     }
104
105     uint64_t h1 = dfs(1, 0, &g1);
106     uint64_t h2 = dfs(1, 0, &g2);
107
108     if (h1 == h2) printf("YES\n");
109     else printf("NO\n");
110
111     free_graph(&g1);
112     free_graph(&g2);
113 }
114 return 0;
115 }
116
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

2

3

1 2

2 3

1 2

1 3

3

1 2

2 3

1 3

3 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree21.exe

NO

YES

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

2

3

1 2

2 3

1 3

3 3

3

1 2

2 3

1 3

3 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree21.exe

NO

NO

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Problem Description

You are given an N-dimensional array A. You can execute an operation that removes the greatest and smallest elements from the array and replaces them with their difference. As a result, the array's size will drop by one after each operation. You are assigned Q jobs, each of which contains an integer K. After K operations, you must provide the total of all the items in the array for each task.

Constraints:

$$2 \leq N \leq 10^5$$

$$1 \leq Q \leq 10^5$$

$$0 \leq A[i] \leq 10^9$$

$$0 \leq K < N$$

Input:

First line contains two space-separated integers **N** and **Q**, denoting the number of elements in array and number of queries respectively.

Next line contains **N** space-separated integers denoting elements of the array.

Next **Q** lines contain a single integer **K**.

Output:

For each task, print answer in a new line.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 2
3 2 1 5 4
1
2

EXPECTED OUTPUT

13
9

Test Case 2

INPUT (STDIN)

5 3
13 12 11 15 14
1
2
3

EXPECTED OUTPUT

43
35
15

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=0;i<n;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

300

Test Case 3

NLOC

38

Tree22.c +

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     long long val;
7     int id;
8 } Node;
9
10 typedef struct {
11     Node *data;
12     int size;
13     int is_min;
14 } Heap;
15
16 void push_heap(Heap *h, long long val, int id) {
17     int i = h->size++;
18     h->data[i].val = val;
19     h->data[i].id = id;
20     while (i > 0) {
21         int p = (i - 1) / 2;
22         int cond = h->is_min ? (h->data[i].val < h->data[p].val) : (h->data[i].val > h->data[p].val);
23         if (cond) {
24             Node tmp = h->data[i];
25             h->data[i] = h->data[p];
26             h->data[p] = tmp;
27             i = p;
28         } else {
29             break;
30         }
31     }
```

```

31 }
32}
33
34Node pop_heap(Heap *h) {
35     Node top = h->data[0];
36     h->data[0] = h->data[--h->size];
37     int i = 0;
38     while (2 * i + 1 < h->size) {
39         int left = 2 * i + 1;
40         int right = 2 * i + 2;
41         int best = left;
42         if (right < h->size) {
43             int cond = h->is_min ? (h->data[right].val < h->data[left].val) : (h->data[right].val > h->data[left].val);
44             if (cond) best = right;
45         }
46         int cond = h->is_min ? (h->data[best].val < h->data[i].val) : (h->data[best].val > h->data[i].val);
47         if (cond) {
48             Node tmp = h->data[i];
49             h->data[i] = h->data[best];
50             h->data[best] = tmp;
51             i = best;
52         } else {
53             break;
54         }
55     }
56     return top;

```

```

57}
58
59int main() {
60    int n, q;
61    if (scanf("%d %d", &n, &q) != 2) return 0;
62    long long *a = (long long*)malloc(n * sizeof(long long));
63    long long initial_sum = 0;
64    int i;
65    for(i=0;i<n;i++) {
66        if (scanf("%lld", &a[i]) != 1) a[i] = 0;
67        initial_sum += a[i];
68    }
69
70    int max_nodes = 2 * n + 5;
71    Heap min_h, max_h;
72    min_h.data = (Node*)malloc(max_nodes * sizeof(Node));
73    min_h.size = 0;
74    min_h.is_min = 1;
75
76    max_h.data = (Node*)malloc(max_nodes * sizeof(Node));
77    max_h.size = 0;
78    max_h.is_min = 0;
79
80    char *deleted = (char*)calloc(max_nodes, sizeof(char));
81

```

main | CH.SC.U4CSE25225 | Tree22.c

Ln 131, Col 1 Spaces: 4 UTF-8 CRLF C

```

82     for (i = 0; i < n; i++) {
83         push_heap(&min_h, a[i], i);
84         push_heap(&max_h, a[i], i);
85     }
86
87     long long *ans = (long long*)malloc(n * sizeof(long long));
88     ans[0] = initial_sum;
89
90     int next_id = n;
91     long long current_sum = initial_sum;
92
93     for (int step = 1; step < n; step++) {
94         while (min_h.size > 0 && deleted[min_h.data[0].id]) {
95             pop_heap(&min_h);
96         }
97         Node mn = pop_heap(&min_h);
98         deleted[mn.id] = 1;
99
100        while (max_h.size > 0 && deleted[max_h.data[0].id]) {
101            pop_heap(&max_h);
102        }
103        Node mx = pop_heap(&max_h);
104        deleted[mx.id] = 1;
105
106        long long diff = mx.val - mn.val;

```

```

106     long long diff = max_val - min_val;
107     current_sum -= 2 * mn.val;
108     ans[step] = current_sum;
109
110     int new_id = next_id++;
111     push_heap(&min_h, diff, new_id);
112     push_heap(&max_h, diff, new_id);
113 }
114
115 for (i = 0; i < q; i++) {
116     int k;
117     if (scanf("%d", &k) == 1) {
118         if (k >= 0 && k < n) {
119             printf("%lld\n", ans[k]);
120         }
121     }
122 }
123
124 free(a);
125 free(min_h.data);
126 free(max_h.data);
127 free(deleted);
128 free(ans);
129 return 0;
130 }
131

```


C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 2

3 2 1 5 4

1

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree22.exe

13

9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3

13 12 11 15 14

1

2

3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree22.exe

43

35

15

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question Description:

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input

The first input line has two integers n and m : the number of cities and flights. The cities are numbered $1, 2, \dots, n$.

After this, there are m lines describing the flights. Each line has two integers a and b : there is a flight from city a to city b . All flights are one-way flights.

Output

First print an integer k : the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Test Case 1

INPUT (STDIN)

4 5
1 2
2 3
3 1
1 4
3 4

EXPECTED OUTPUT

1
4 1

Test Case 2

INPUT (STDIN)

4 5
1 3
2 1
1 2
2 4
1 4

EXPECTED OUTPUT

2
4 3
3 1

✓ Mandatory Test Cases

Test Case 1

KEYWORD

`while(m--)`

✓ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

1080

Test Case 3

NLOC

130

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     int *head;
7     int *to;
8     int *nxt;
9     int edge_cnt;
10} Adj;
11
12void init_adj(Adj *a, int n, int max_edges) {
13    a->head = (int*)malloc((n + 1) * sizeof(int));
14    for (int i = 0; i <= n; i++) a->head[i] = -1;
15    a->to = (int*)malloc(max_edges * sizeof(int));
16    a->nxt = (int*)malloc(max_edges * sizeof(int));
17    a->edge_cnt = 0;
18}
19
20void add_dir_edge(Adj *a, int u, int v) {
21    a->to[a->edge_cnt] = v;
22    a->nxt[a->edge_cnt] = a->head[u];
23    a->head[u] = a->edge_cnt++;
24}
25
```

```

26void free_adj(Adj *a) {
27    free(a->head);
28    free(a->to);
29    free(a->nxt);
30}
31
32int n, m;
33int *order, order_cnt;
34int *visited;
35
36void dfs1(int u, Adj *g) {
37    visited[u] = 1;
38    for (int e = g->head[u]; e != -1; e = g->nxt[e]) {
39        int v = g->to[e];
40        if (!visited[v]) dfs1(v, g);
41    }
42    order[order_cnt++] = u;
43}
44
45int *scc;
46int scc_cnt;
47int *rep;
48

```

```

49void dfs2(int u, int c, Adj *rg) {
50    scc[u] = c;
51    for (int e = rg->head[u]; e != -1; e = rg->nxt[e]) {
52        int v = rg->to[e];
53        if (scc[v] == 0) dfs2(v, c, rg);
54    }
55}
56
57int *s_in, *s_out;
58int *matched_snk;
59int *vis_dag;
60
61int dfs_match(int u, Adj *dag, int *snk_visited) {
62    vis_dag[u] = 1;
63    if (s_out[u] == 0) {
64        if (!snk_visited[u]) {
65            snk_visited[u] = 1;
66            return u;
67        }
68    }
69    for (int e = dag->head[u]; e != -1; e = dag->nxt[e]) {
70        int v = dag->to[e];

```

```

71     if (!vis_dag[v]) {
72         int res = dfs_match(v, dag, snk_visited);
73         if (res != 0) return res;
74     }
75 }
76 return 0;
77}
78
79int main() {
80     if (scanf("%d %d", &n, &m) != 2) return 0;
81     Adj g, rg;
82     init_adj(&g, n, m + 5);
83     init_adj(&rg, n, m + 5);
84
85     int orig_m = m;
86     while(m--) {
87         int u, v;
88         if (scanf("%d %d", &u, &v) == 2) {
89             add_dir_edge(&g, u, v);
90             add_dir_edge(&rg, v, u);
91         }
92     }

```

```
93
94  order = (int*)malloc((n + 1) * sizeof(int));
95  order_cnt = 0;
96  visited = (int*)calloc(n + 1, sizeof(int));
97
98  for (int i = 1; i <= n; i++) {
99      if (!visited[i]) dfs1(i, &g);
100  }
101
```

main | CH.SC.U4CSE25225 | Tree23.c

Ln 202, Col 1 Spaces: 4 UTF-8 CRLF C

```
103  scc_cnt = 0;
104  rep = (int*)malloc((n + 1) * sizeof(int));
105
106  for (int i = n - 1; i >= 0; i--) {
107      int u = order[i];
108      if (scc[u] == 0) {
109          scc_cnt++;
110          rep[scc_cnt] = u;
111          dfs2(u, scc_cnt, &rg);
112      }
113  }
114
```

```

115     if (scc_cnt == 1) {
116         printf("0\n");
117         return 0;
118     }
119
120     Adj dag;
121     init_adj(&dag, scc_cnt, orig_m + 5);
122     s_in = (int*)calloc(scc_cnt + 1, sizeof(int));
123     s_out = (int*)calloc(scc_cnt + 1, sizeof(int));
124
125     for (int u = 1; u <= n; u++) {
126         for (int e = g.head[u]; e != -1; e = g.nxt[e]) {
127             int v = g.to[e];
128             if (scc[u] != scc[v]) {
129                 add_dir_edge(&dag, scc[u], scc[v]);
130                 s_out[scc[u]]++;
131                 s_in[scc[v]]++;
132             }
133         }
134     }
135
136     int *sources = (int*)malloc((scc_cnt + 1) * sizeof(int));

```

```

137 int num_sources = 0;
138 int *sinks = (int*)malloc((scc_cnt + 1) * sizeof(int));
139 int num_sinks = 0;
140
141 for (int i = 1; i <= scc_cnt; i++) {
142     if (s_in[i] == 0) sources[num_sources++] = i;
143     if (s_out[i] == 0) sinks[num_sinks++] = i;
144 }
145
146 vis_dag = (int*)malloc((scc_cnt + 1) * sizeof(int));
147 int *snk_visited = (int*)calloc(scc_cnt + 1, sizeof(int));
148
149 int *pair_src = (int*)malloc((scc_cnt + 1) * sizeof(int));
150 int *pair_snk = (int*)malloc((scc_cnt + 1) * sizeof(int));
151 int pair_cnt = 0;
152
153 for (int i = 0; i < num_sources; i++) {
154     for (int j = 1; j <= scc_cnt; j++) vis_dag[j] = 0;
155     int target_snk = dfs_match(sources[i], &dag, snk_visited);
156     if (target_snk != 0) {
157         pair_src[pair_cnt] = sources[i];
158         pair_snk[pair_cnt] = target_snk;

```

```

159         pair_cnt++;
160     }
161 }
162
163 int *unpaired_src = (int*)malloc((num_sources + 1) * sizeof(int));
164 int unp_src_cnt = 0;
165 int *src_is_paired = (int*)calloc(scc_cnt + 1, sizeof(int));
166 for (int i = 0; i < pair_cnt; i++) src_is_paired[pair_src[i]] = 1;
167 for (int i = 0; i < num_sources; i++) {
168     if (!src_is_paired[sources[i]]) unpaired_src[unp_src_cnt++] = sources[i];
169 }
170
171 int *unpaired_snk = (int*)malloc((num_sinks + 1) * sizeof(int));
172 int unp_snk_cnt = 0;
173 for (int i = 0; i < num_sinks; i++) {
174     if (!snk_visited[sinks[i]]) unpaired_snk[unp_snk_cnt++] = sinks[i];
175 }
176
177 int *all_ordered_src = (int*)malloc((num_sources + num_sinks + 5) * sizeof(int));
178 int *all_ordered_snk = (int*)malloc((num_sources + num_sinks + 5) * sizeof(int));
179
180 for (int i = 0; i < pair_cnt; i++) {

```

```

181     all_ordered_src[i] = pair_src[i];
182     all_ordered_snk[i] = pair_snk[i];
183 }
184
185 int cur_s = pair_cnt;
186 for (int i = 0; i < unp_src_cnt; i++) all_ordered_src[cur_s++] = unpaired_src[i];
187
188 int cur_t = pair_cnt;
189 for (int i = 0; i < unp_snk_cnt; i++) all_ordered_snk[cur_t++] = unpaired_snk[i];
190
191 int total_ans = num_sources > num_sinks ? num_sources : num_sinks;
192 printf("%d\n", total_ans);
193
194 for (int i = 0; i < total_ans; i++) {
195     int u_scc = all_ordered_snk[i % num_sinks];
196     int v_scc = all_ordered_src[(i + 1) % num_sources];
197     printf("%d %d\n", rep[u_scc], rep[v_scc]);
198 }
199
200 return 0;
201 }
202

```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4 5

1 2

2 3

3 1

1 4

3 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree23.exe

1

4 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 5

1 3

2 1

1 2

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree23.exe

2

4 1

3 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Problem Description

You are given a weighted graph with **N** vertices and **M** edges. Find the total weight of its **maximum** spanning tree.

Constraints

- $1 \leq T \leq 20$
- $1 \leq N \leq 5000$
- $1 \leq M \leq 100\,000$
- $1 \leq c \leq 10\,000$

Input

The first line contains one integer **T** denoting the number of test cases. Each test case starts with a line containing **2** space-separated integers: **N** and **M**. Each of the following **M** lines contain description of one edge: three different space-separated integers: **a**, **b** and **c**. **a** and **b** are different and from 1 to N each and denote numbers of vertices that are connected by this edge. **c** denotes weight of this edge.

Output

For each test case output one integer - the total weight of its **maximum** spanning tree.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

1
3 3
1 2 2
2 3 3
1 3 4

EXPECTED OUTPUT

7

Test Case 2

INPUT (STDIN)

1
3 3
1 2 3
2 3 4
1 3 5

EXPECTED OUTPUT

9

▼ Mandatory Test Cases

Test Case 1

KEYWORD

int printheap(int N)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

715

Test Case 3

NLOC

110

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     int u, v, w;
7 } Edge;
8
9 int cmp_edge(const void *a, const void *b) {
10     Edge *e1 = (Edge*)a;
11     Edge *e2 = (Edge*)b;
12     return e2->w - e1->w;
13}
14
15int find_set(int *parent, int i) {
16     if (parent[i] == i)
17         return i;
18     return parent[i] = find_set(parent, parent[i]);
19}
20
21void union_sets(int *parent, int *rank, int i, int j) {
22     int root_i = find_set(parent, i);
23     int root_j = find_set(parent, j);
24     if (root_i != root_j) {
25         if (rank[root_i] < rank[root_j]) {
```

```
26     parent[root_i] = root_j;
27     } else if (rank[root_i] > rank[root_j]) {
28         parent[root_j] = root_i;
29     } else {
30         parent[root_j] = root_i;
31         rank[root_i]++;
32     }
33 }
34}
35
36int printheap(int N) {
37     return N;
38}
39
```

```
int main() {
```

main | CH.SC.U4CSE25225 | Tree24.c

Ln 80, Col 1 Spaces: 4 UTF-8 CRLF C

```
42     if (scanf("%d", &t) != 1) return 0;
43     while (t--) {
44         int n, m;
45         if (scanf("%d %d", &n, &m) != 2) break;
46         Edge *edges = (Edge*)malloc(m * sizeof(Edge));
47         for (int i = 0; i < m; i++) {
48             if (scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w) != 3) {
49                 edges[i].u = edges[i].v = edges[i].w = 0;
50             }
51         }
52
53         qsort(edges, m, sizeof(Edge), cmp_edge);
```

```
54
55     int *parent = (int*)malloc((n + 1) * sizeof(int));
56     int *rank = (int*)calloc(n + 1, sizeof(int));
57     for (int i = 1; i <= n; i++) parent[i] = i;
58
59     long long total_weight = 0;
60     int edges_count = 0;
61
62     for (int i = 0; i < m && edges_count < n - 1; i++) {
63         int u = edges[i].u;
64         int v = edges[i].v;
65         if (find_set(parent, u) != find_set(parent, v)) {
66             union_sets(parent, rank, u, v);
67             total_weight += edges[i].w;
```

```
68         edges_count++;
69     }
70 }
71
72     printf("%lld\n", total_weight);
73
74     free(edges);
75     free(parent);
76     free(rank);
77 }
78 return 0;
79 }
80
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

1
3 3
1 2 2
2 3 3
1 3 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree24.exe

7

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

1
3 3
1 2 3
2 3 4
1 3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree24.exe

9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

A beautiful *code* of a tree of n nodes is a sequence of $n-2$ integers that uniquely specifies the structure of the tree.

The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbour to the code, and remove the leaf from the tree.

Given a beautiful code of a tree, your task is to construct the original tree.

Constraints

- $3 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Input

The first input line contains an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

The second line contains $n-2$ integers: the beautiful code.

Output

Print $n-1$ lines describing the edges of the tree. Each line has to contain two integers a and b : there is an edge between nodes a and b . You can print the edges in any order.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
2 2 4

EXPECTED OUTPUT

1 2
3 2
2 4
4 5

Test Case 2

INPUT (STDIN)

5
3 1 2

EXPECTED OUTPUT

4 3
3 1
1 2
2 5

▼ Mandatory Test Cases

Test Case 1

KEYWORD

q.pop();

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

300

Test Case 3

NLOC

34

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     int *data;
7     int size;
8 } MinHeap;
9
10 void push_heap(MinHeap *h, int val) {
11     int i = h->size++;
12     h->data[i] = val;
13     while (i > 0) {
14         int p = (i - 1) / 2;
15         if (h->data[i] < h->data[p]) {
16             int tmp = h->data[i];
17             h->data[i] = h->data[p];
18             h->data[p] = tmp;
19             i = p;
20         } else {
21             break;
22         }
23     }
24 }
25
```

```

26 int pop_heap(MinHeap *h) {
27     int top = h->data[0];
28     h->data[0] = h->data[--h->size];
29     int i = 0;
30     while (2 * i + 1 < h->size) {
31         int left = 2 * i + 1;
32         int right = 2 * i + 2;
33         int best = left;
34         if (right < h->size && h->data[right] < h->data[left]) {
35             best = right;
36         }
37         if (h->data[best] < h->data[i]) {
38             int tmp = h->data[i];
39             h->data[i] = h->data[best];
40             h->data[best] = tmp;
41             i = best;
42         } else {
43             break;
44         }

```

```

45     }
46     return top;
47 }
48
49 void dummy_pop(void) {}
50

```

main | CH.SC.U4CSE25225 | Tree25.c

Ln 99, Col 1 Spaces: 4 UTF-8 CRLF C

```

51 struct {
52     void (*pop)(void);
53 } q = { dummy_pop };
54
55 int main() {
56     q.pop();
57     int n;
58     if (scanf("%d", &n) != 1) return 0;
59     int *code = (int*)malloc((n - 2) * sizeof(int));
60     int *deg = (int*)malloc((n + 1) * sizeof(int));
61     for (int i = 1; i <= n; i++) deg[i] = 1;
62
63     for (int i = 0; i < n - 2; i++) {

```

```
64     if (scanf("%d", &code[i]) == 1) {
65         deg[code[i]]++;
66     }
67 }
68
69 MinHeap h;
70 h.data = (int*)malloc((n + 5) * sizeof(int));
71 h.size = 0;
72
73 for (int i = 1; i <= n; i++) {
74     if (deg[i] == 1) {
75         push_heap(&h, i);
76     }
77 }
78
79 for (int i = 0; i < n - 2; i++) {
80     int leaf = pop_heap(&h);
81     int c = code[i];
```

```
82     printf("%d %d\n", leaf, c);
83     deg[leaf]--;
84     deg[c]--;
85     if (deg[c] == 1) {
86         push_heap(&h, c);
87     }
88 }
89
90 int u = pop_heap(&h);
91 int v = pop_heap(&h);
92 printf("%d %d\n", u, v);
93
94 free(code);
95 free(deg);
96 free(h.data);
97 return 0;
98}
99
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

2 2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree25.exe

1 2

3 2

2 4

4 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5

3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree25.exe

4 3

3 1

1 2

2 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range $[a,b]$ by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range $[a,b]$.

Input

The first input line has two integers n and q : the size of the array and the number of queries.

The next line has n values $t_1, t_2, \dots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either "1 a b " or "2 a b ".

Output

Print the answer to each sum query.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq t_i \leq 10^6$
- $1 \leq a \leq b \leq n$

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3

EXPECTED OUTPUT

9

Test Case 2

INPUT (STDIN)

5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5

EXPECTED OUTPUT

17
32

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void build(int k,int
l,int r)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

800

Test Case 3

NLOC

71

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define MAXN 200005
6
7 long long tree_sum[4 * MAXN];
8 long long lazy_a[4 * MAXN];
9 long long lazy_d[4 * MAXN];
10 long long initial_arr[MAXN];
11
12 void build(int k, int l, int r) {
13     lazy_a[k] = 0;
14     lazy_d[k] = 0;
15     if (l == r) {
16         tree_sum[k] = initial_arr[l];
17         return;
18     }
19     int mid = (l + r) / 2;
20     build(2 * k, l, mid);
21     build(2 * k + 1, mid + 1, r);
22     tree_sum[k] = tree_sum[2 * k] + tree_sum[2 * k + 1];
23 }
24
25 void apply_lazy(int k, int l, int r, long long a, long long d) {
```

```

26     long long len = r - l + 1;
27     tree_sum[k] += len * a + d * len * (len - 1) / 2;
28     lazy_a[k] += a;
29     lazy_d[k] += d;
30 }
31
32 void push(int k, int l, int r) {
33     if (lazy_a[k] != 0 || lazy_d[k] != 0) {
34         int mid = (l + r) / 2;
35         apply_lazy(2 * k, l, mid, lazy_a[k], lazy_d[k]);
36         apply_lazy(2 * k + 1, mid + 1, r, lazy_a[k] + (mid + 1 - l) * lazy_d[k], lazy_d[k]);
37         lazy_a[k] = 0;
38         lazy_d[k] = 0;
39     }
40 }
41

```

```
42 void update(int k, int l, int r, int ql, int qr) {
```

```
43     if (ql <= l && r <= qr) {
```

```
45         long long d = 1;
```

```
46         apply_lazy(k, l, r, a, d);
```

```
47         return;
```

```
48     }
```

```
49     push(k, l, r);
```

```
50     int mid = (l + r) / 2;
```

```
51     if (ql <= mid) update(2 * k, l, mid, ql, qr);
```

```
52     if (qr > mid) update(2 * k + 1, mid + 1, r, ql, qr);
```

```
53     tree_sum[k] = tree_sum[2 * k] + tree_sum[2 * k + 1];
```

```
54 }
```

```
55
```

```
56 long long query(int k, int l, int r, int ql, int qr) {
```

```
57     if (ql <= l && r <= qr) {
58         return tree_sum[k];
59     }
60     push(k, l, r);
61     int mid = (l + r) / 2;
62     long long res = 0;
63     if (ql <= mid) res += query(2 * k, l, mid, ql, qr);
64     if (qr > mid) res += query(2 * k + 1, mid + 1, r, ql, qr);
65     return res;
66 }
67
68 int main() {
69     int n, q;
70     if (scanf("%d %d", &n, &q) != 2) return 0;
71     for (int i = 1; i <= n; i++) {
```

```
72     if (scanf("%lld", &initial_arr[i]) != 1) initial_arr[i] = 0;
73 }
74 build(1, 1, n);
75 while (q--) {
76     int type, a, b;
77     if (scanf("%d %d %d", &type, &a, &b) != 3) break;
78     if (type == 1) {
79         update(1, 1, n, a, b);
80     } else {
81         printf("%lld\n", query(1, 1, n, a, b));
82     }
83 }
84 return 0;
85}
86
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 3

3 7 2 2 5

3 2 3

1 3 6

1 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree26.exe

9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3

4 2 3 1 7

2 1 5

1 1 5

2 1 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree26.exe

17

32

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Problem Description:

Any sequence A of size n is called **B-sequence** if:

$A_1 < A_2 < \dots < A_k > A_{k+1} > A_{k+2} > \dots > A_n$ where $1 \leq k \leq n$. That is, a sequence which is initially strictly increasing and then strictly decreasing (the decreasing part may or may not be there).

All elements in A except the maximum element comes at most twice (once in increasing part and once in decreasing part) and maximum element comes exactly once.

All elements coming in decreasing part of sequence should have come once in the increasing part of sequence.

You are given a B-sequence S and Q operations. For each operation, you are given a value **val**. You have to insert **val** in S if and only if after insertion, S still remains a B-sequence.

After each operation, print the size of S . After all the operations, print the sequence S .

Hint: Think of using some data structure to support insertion of elements in complexity better than linear.

Input Constraints:

$$1 \leq N \leq 10^5$$

$$1 \leq S_i \leq 10^9$$

$$1 \leq Q \leq 10^5$$

$$1 \leq val \leq 10^9$$

Given sequence S is a **B-sequence**,

Input Format:

First line consists of an integer N , denoting size of S .

Second line consists of N space separated integers, denoting elements of S .

Next line consists of an integer **Q**, denoting number of operations.

Each of the following **Q** lines consists of an integer **val**.

Output Format:

After each operation, print the size of **S** in a new line.

After all operations, print the sequence **S**.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

4
1 2 5 2
4
5
1
3
2

EXPECTED OUTPUT

4
5
6
6
1 2 3 5 2 1

Test Case 2

INPUT (STDIN)

5
3 4 5 1 3
3
6
2
3
1

EXPECTED OUTPUT

6
7
7
1 2 3 4 5 6 3

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=0;i<N;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

39

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define HASH_SIZE 1000003
6
7 typedef struct {
8     int val;
9     int count;
10    int used;
11} HashEntry;
12
13 HashEntry hash_table[HASH_SIZE];
14
15 int find_slot(int val) {
16     unsigned int h = ((unsigned int)val * 2654435761U) % HASH_SIZE;
17     while (hash_table[h].used && hash_table[h].val != val) {
18         h = (h + 1) % HASH_SIZE;
19     }
20     return h;
21 }
22
23 int get_count(int val) {
24     int slot = find_slot(val);
25     if (!hash_table[slot].used) return 0;
```

```

26     return hash_table[slot].count;
27 }
28
29 void set_count(int val, int count) {
30     int slot = find_slot(val);
31     hash_table[slot].val = val;
32     hash_table[slot].count = count;
33     hash_table[slot].used = 1;
34 }
35
36 int cmp_int(const void *a, const void *b) {
37     int x = *(const int*)a;
38     int y = *(const int*)b;
39     if (x < y) return -1;
40     if (x > y) return 1;
41     return 0;
42 }
43
44 int main() {
45     int N;

```

```
46     if (scanf("%d", &N) != 1) return 0;
47
48     int *init_s = (int*)malloc(N * sizeof(int));
49     int i;
50     for(i=0;i<N;i++) {
51         if (scanf("%d", &init_s[i]) != 1) init_s[i] = 0;
52     }
53
54     int *unique_vals = (int*)malloc(300005 * sizeof(int));
55     int unique_cnt = 0;
56     int current_size = N;
57     int max_val = init_s[0];
58
59     for (i = 0; i < N; i++) {
60         int v = init_s[i];
61         if (v > max_val) max_val = v;
```

main | CH.SC.U4CSE25225 | Tree27.c

Ln 122, Col 1 Spaces: 4 UTF-8 CRLF C

```
63         if (c == 0) {
64             unique_vals[unique_cnt++] = v;
65         }
```

```
66     set_count(v, c + 1);
67 }
68
69 int Q;
70 if (scanf("%d", &Q) != 1) Q = 0;
71
72 while (Q--) {
73     int val;
74     if (scanf("%d", &val) != 1) break;
75     if (val > max_val) {
76         int c = get_count(val);
77         if (c == 0) unique_vals[unique_cnt++] = val;
78         set_count(val, 1);
79         max_val = val;
80         current_size++;
81     } else if (val == max_val) {
82     } else {
83         int c = get_count(val);
84         if (c == 0) {
```

```

85         unique_vals[unique_cnt++] = val;
86         set_count(val, 1);
87         current_size++;
88     } else if (c == 1) {
89         set_count(val, 2);
90         current_size++;
91     }
92 }
93 printf("%d\n", current_size);
94 }
95
96 qsort(unique_vals, unique_cnt, sizeof(int), cmp_int);
97
98 int first = 1;
99 for (i = 0; i < unique_cnt; i++) {
100     int v = unique_vals[i];
101     if (get_count(v) >= 1) {
102         if (!first) printf(" ");
103         printf("%d", v);

```

```
104         first = 0;
105     }
106 }
107
108 for (i = unique_cnt - 1; i >= 0; i--) {
109     int v = unique_vals[i];
110     if (get_count(v) == 2 && v != max_val) {
111         if (!first) printf(" ");
112         printf("%d", v);
113         first = 0;
114     }
115 }
116 printf("\n");
117
118 free(init_s);
119 free(unique_vals);
120 return 0;
121 }
122
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4

1 2 5 2

4

5

1

3

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree27.exe

4

5

6

6

1 2 3 5 2 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5

3 4 5 1 3

3

6

2

3

1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree27.exe

6

7

7

1 2 3 4 5 6 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Problem description

When the king of ghosts notices that all humans on Planet Earth have lost their dread of the ghost race, he is extremely unhappy. He understands why this is happening. The existing ghost species has gotten incredibly lethargic and has ceased visiting Planet Earth in order to terrorise humanity. As a result, he plans to arrange a tournament to urge the whole ghost race to scare the humans. The monarch, on the other hand, never comes to Planet Earth.

This competition will go on for **N days**. Currently, there are a total of **M ghosts** (apart from the king) existing in the ghost race such that :

- The youngest ghost is 1 year old.
- The oldest ghost is M years old.
- No two ghosts have the same age.
- The age of each and every ghost is a positive integer.

Every day of the tournament, ghosts must visit Planet Earth in order to frighten humans. The ghost that terrifies the most humans on any given day is named "Ghost of the Day" at the conclusion of the day. The king of ghosts, on the other hand, is a firm believer in constancy. After this title is granted, a "Consistency Trophy" is offered to the ghost who has earned the most of these titles up until that point. If there are several such ghosts, the prize is handed to the eldest among them. It's worth noting that "Title Giving" and "Trophy Giving" take place at the end of each tournament day.

Each day of the competition, you will be told the age of the ghost that earned the "Ghost of the Day" title. Your task is to determine the age of the ghost who received the "Consistency Trophy" on each competition day.

Input

The first line consists of 2 space separated integers **N** and **M**. The next line consists of **N** space separated integers such that the i^{th} integer denotes the age of the ghost who was awarded with the **"Ghost of the Day" title** on the i^{th} day of the competition.

Output

Print **N** lines. The i^{th} line should contain 2 space separated integers such that the first integer denotes the **age of the ghost** who was awarded with the

"Consistency Trophy" on the i^{th} day and the second integer denotes the number of "Ghost of the Day" titles won by this ghost until the end of the i^{th} day of the competition.

Constraints

$1 \leq N \leq 10^5$

$1 \leq M \leq 10^9$

Logical Test Cases

Test Case 1

INPUT (STDIN)

7 5
1 3 1 3 2 2 2

EXPECTED OUTPUT

1 1
3 1
1 2
3 2
3 2
3 2
2 3

Test Case 2

INPUT (STDIN)

10 4
3 4 5 1 3 1 3 2 2 2

EXPECTED OUTPUT

3 1
4 1
5 1
5 1
3 2
3 2
3 3
3 3
3 3
3 3

Mandatory Test Cases

Test Case 1

KEYWORD

for(i = 0;i<n-1;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

5

Test Case 2

TOKEN COUNT

190

Test Case 3

NLOC

28

Tree28.c +

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define MAX_GHOST 100005
6
7 typedef struct {
8     int count;
9     int age;
10} HeapNode;
11
12typedef struct {
13     HeapNode *data;
14     int size;
15} MaxHeap;
16
17void push_heap(MaxHeap *h, int count, int age) {
18     int i = h->size++;
19     h->data[i].count = count;
20     h->data[i].age = age;
21     while (i > 0) {
22         int p = (i - 1) / 2;
23         int better = 0;
24         if (h->data[i].count > h->data[p].count) better = 1;
25         else if (h->data[i].count == h->data[p].count && h->data[i].age > h->data[p].age) better = 1;
26         if (better) {
27             HeapNode tmp = h->data[i];
28             h->data[i] = h->data[p];
29             h->data[p] = tmp;
30             i = p;
31         } else {
32             break;
33         }
34     }
35 }
```

```

32     }
33 }
34 }
35}
36
37void pop_heap(MaxHeap *h) {
38    h->data[0] = h->data[--h->size];
39    int i = 0;
40    while (2 * i + 1 < h->size) {
41        int left = 2 * i + 1;
42        int right = 2 * i + 2;
43        int best = left;
44        if (right < h->size) {
45            int right_better = 0;
46            if (h->data[right].count > h->data[left].count) right_better = 1;
47            else if (h->data[right].count == h->data[left].count && h->data[right].age > h->data[left].age) right_better = 1;
48            if (right_better) best = right;
49        }
50        int best_better = 0;
51        if (h->data[best].count > h->data[i].count) best_better = 1;
52        else if (h->data[best].count == h->data[i].count && h->data[best].age > h->data[i].age) best_better = 1;
53        if (best_better) {
54            HeapNode tmp = h->data[i];
55            h->data[i] = h->data[best];
56            h->data[best] = tmp;
57        }
58    }
59}

```

```

57         i = best;
58     } else {
59         break;
60     }
61 }
62}
63
64int count_arr[MAX_GHOST];
65
66int main() {
67     int n, m;
68     if (scanf("%d %d", &n, &m) != 2) return 0;
69
70     int *ages = (int*)malloc(n * sizeof(int));
71     int i;
72     for (i = 0; i < n; i++) {
73         if (scanf("%d", &ages[i]) != 1) ages[i] = 0;
74     }
75
76     MaxHeap h;
77     h.data = (HeapNode*)malloc((n + 5) * sizeof(HeapNode));
78     h.size = 0;
79
80     for(i = 0; i < n - 1; i++) {
81         int age = ages[i];

```

```

82     count_arr[age]++;
83     push_heap(&h, count_arr[age], age);
84     while (h.size > 0 && h.data[0].count != count_arr[h.data[0].age]) {
85         pop_heap(&h);
86     }
87     printf("%d %d\n", h.data[0].age, h.data[0].count);
88 }
89 if (n > 0) {
90     int age = ages[n - 1];
91     count_arr[age]++;
92     push_heap(&h, count_arr[age], age);
93     while (h.size > 0 && h.data[0].count != count_arr[h.data[0].age]) {
94         pop_heap(&h);
95     }
96     printf("%d %d\n", h.data[0].age, h.data[0].count);
97 }
98
99 free(ages);
100 free(h.data);
101 return 0;
102
103

```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
7 5
```

```
1 3 1 3 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree28.exe
```

```
1 1
```

```
3 1
```

```
1 2
```

```
3 2
```

```
3 2
```

```
3 2
```

```
2 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
10 4
```

```
3 4 5 1 3 1 3 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree28.exe
```

```
3 1
```

```
4 1
```

```
5 1
```

```
5 1
```

```
3 2
```

```
3 2
```

```
3 3
```

```
3 3
```

```
3 3
```

```
3 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Problem Description:

You are given a rooted tree that contains N nodes. Each node contains a lowercase alphabet.

You are required to answer Q queries of type u, c , where u is an integer and c is a lowercase alphabet. The count of nodes in the subtree of the node u containing c is considered as the answer of all the queries.

Constraints

$$1 \leq N, Q \leq 10^5$$

$$1 \leq u, v \leq N$$

- c is a lowercase alphabet
- s_i is a lowercase alphabet for all $1 \leq i \leq N$
- 1 is the root node

Input format

- First line: Two space-separated integers N and Q respectively
- Second line: A string s of length N (where the i^{th} character of s represents the character stored in node i)
- Next $N - 1$ line: Two space-separated integers u and v denoting an edge between node u and node v
- Next Q lines: An integer u and a space-separated character c

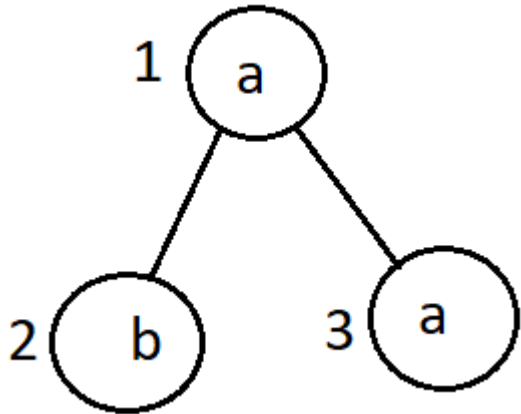
Output format

For each query, print the output in a new line.

Note: It is guaranteed that the input generates a valid tree.

Explanation for test case 1

Tree given in the sample input will look like that



Number of nodes in the subtree of node 1 having 'a' stored in it is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

3 1
aba
1 2
1 3
1 a

EXPECTED OUTPUT

2

Test Case 2

INPUT (STDIN)

4 1
wsx
1 2
1 3
1 1
1 w

EXPECTED OUTPUT

10

Mandatory Test Cases

Test Case 1

KEYWORD

while(0--)

Test Case 2

KEYWORD

for(i = 0; i < N-1; i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

330

Test Case 3

NLOC

49

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 typedef struct {
7     int *head;
8     int *to;
9     int *nxt;
10    int edge_cnt;
11} Graph;
12
13void init_graph(Graph *g, int n) {
14    g->head = (int*)malloc((n + 1) * sizeof(int));
15    for (int i = 0; i <= n; i++) g->head[i] = -1;
16    g->to = (int*)malloc(2 * (n + 5) * sizeof(int));
17    g->nxt = (int*)malloc(2 * (n + 5) * sizeof(int));
18    g->edge_cnt = 0;
19}
20
21void add_edge(Graph *g, int u, int v) {
22    g->to[g->edge_cnt] = v;
23    g->nxt[g->edge_cnt] = g->head[u];
24    g->head[u] = g->edge_cnt++;
25}
```

```

26
27int timer = 0;
28int *tin, *tout;
29int *char_list[26];
30int char_count[26];
31int char_cap[26];
32char *s;
33
34void dfs(int u, int p, Graph *g) {
35    tin[u] = ++timer;
36    int c = s[u - 1] - 'a';
37    if (c >= 0 && c < 26) {
38        if (char_count[c] >= char_cap[c]) {
39            char_cap[c] = char_cap[c] == 0 ? 16 : char_cap[c] * 2;
40            char_list[c] = (int*)realloc(char_list[c], char_cap[c] * sizeof(int));
41        }
42        char_list[c][char_count[c]++] = tin[u];
43    }
44    for (int e = g->head[u]; e != -1; e = g->nxt[e]) {
45        int v = g->to[e];
46        if (v != p) {

```

```

47         dfs(v, u, g);
48     }
49 }
50 tout[u] = timer;
51}
52
53int lower_bound(int *arr, int n, int val) {
54     int l = 0, r = n;
55     while (l < r) {
56         int mid = (l + r) / 2;
57         if (arr[mid] >= val) r = mid;
58         else l = mid + 1;
59     }
60     return l;
61}
62
63int upper_bound(int *arr, int n, int val) {
64     int l = 0, r = n;

```

main | CH.SC.U4CSE25225 | Tree29.c

Ln 128, Col 1 Spaces: 4 UTF-8 CRLF C

```

66         int mid = (l + r) / 2;
67         if (arr[mid] > val) r = mid;

```

```

68         else l = mid + 1;
69     }
70     return l;
71}
72
73int main() {
74     int N, Q;
75     if (scanf("%d %d", &N, &Q) != 2) return 0;
76
77     s = (char*)malloc((N + 5) * sizeof(char));
78     if (scanf("%s", s) != 1) return 0;
79
80     Graph g;
81     init_graph(&g, N);
82
83     int i;
84     for(i = 0; i < N-1; i++) {
85         int u, v;
86         if (scanf("%d %d", &u, &v) == 2) {
87             add_edge(&g, u, v);
88             add_edge(&g, v, u);

```

```

89     }
90 }
91
92 tin = (int*)malloc((N + 1) * sizeof(int));
93 tout = (int*)malloc((N + 1) * sizeof(int));
94 for (int c = 0; c < 26; c++) {
95     char_list[c] = NULL;
96     char_count[c] = 0;
97     char_cap[c] = 0;
98 }
99
100 dfs(1, 0, &g);
101
102 while(Q--) {
103     int u;
104     char ch[10];
105     if (scanf("%d %s", &u, ch) == 2) {
106         int c = ch[0] - 'a';
107         if (c < 0 || c >= 26) {
108             printf("0\n");

```

```
109         } else {
110             int l = lower_bound(char_list[c], char_count[c], tin[u]);
111             int r = upper_bound(char_list[c], char_count[c], tout[u]);
112             printf("%d\n", r - l);
113         }
114     }
115 }
116
117 free(s);
118 free(tin);
119 free(tout);
120 for (int c = 0; c < 26; c++) {
121     if (char_list[c]) free(char_list[c]);
122 }
123 free(g.head);
124 free(g.to);
125 free(g.nxt);
126 return 0;
127
128
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 1

aba

1 2

1 3

1 a

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree29.exe

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 1

wsx

1 2

1 3

1 1

1 w

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree29.exe

1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Problem Description

Football is Monk's favourite sport, and his favourite team is "Manchester United." Manchester United has qualified for the Champions League Final, which will take place at London's Wembley Stadium. As a result, he decided to go watch his favourite team play.

When he arrived at the stadium, he noticed that there was a long wait for match tickets. He is aware that the stadium has M rows, each with a distinct seating capacity. They could or might not be comparable. The cost of a ticket is determined by the row. If there are K (always higher than 0) empty seats in a row, the ticket will cost K pounds (units of British Currency).

Now, every football fan standing in the line will get a ticket one by one.

Given the seating capacities of different rows, find the maximum possible pounds that the club will gain with the help of the ticket sales.

Constraints:

$$1 \leq M \leq 1000000$$

$$1 \leq N \leq 1000000$$

$$1 \leq X[i] \leq 1000000$$

Sum of $X[i]$ for all $1 \leq i \leq M$ will always be greater than N .

Input:

The first line consists of M and N . M denotes the number of seating rows in the stadium and N denotes the number of football fans waiting in the line to get a ticket for the match.

Next line consists of M space separated integers $X[1], X[2], X[3], \dots, X[M]$ where $X[i]$ denotes the number of empty seats initially in the i^{th} row.

Output:

Print in a single line the maximum pounds the club will gain.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3 4
1 2 4

EXPECTED OUTPUT

11

Test Case 2

INPUT (STDIN)

8 4
3 7 1 2 4 7 5 9

EXPECTED OUTPUT

31

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void heapify(int
arr[],int n,int i)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

450

Test Case 3

NLOC

70

Tree210.c +

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 void heapify(int arr[],int n,int i) {
6     int largest = i;
7     int left = 2 * i + 1;
8     int right = 2 * i + 2;
9
10    if (left < n && arr[left] > arr[largest])
11        largest = left;
12    if (right < n && arr[right] > arr[largest])
13        largest = right;
14
15    if (largest != i) {
16        int temp = arr[i];
17        arr[i] = arr[largest];
18        arr[largest] = temp;
19        heapify(arr, n, largest);
20    }
21}
22
23int main() {
24    int m, n;
25    if (scanf("%d %d", &m, &n) != 2) return 0;
```

```
26
27  int *arr = (int*)malloc(m * sizeof(int));
28  for (int i = 0; i < m; i++) {
29      if (scanf("%d", &arr[i]) != 1) arr[i] = 0;
30  }
31
32  for (int i = m / 2 - 1; i >= 0; i--) {
33      heapify(arr, m, i);
34  }
35
36  long long total_gain = 0;
37  for (int i = 0; i < n; i++) {
```

```
38     if (arr[0] <= 0) break;
39     total_gain += arr[0];
40     arr[0]--;
41     heapify(arr, m, 0);
42 }
43
44 printf("%lld\n", total_gain);
45
46 free(arr);
47 return 0;
48}
49
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 4

1 2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree210.exe

11

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

8 4

3 7 1 2 4 7 5 9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree210.exe

31

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>